

守灯 ShouDeng

紧急提示系统 – 技术架构文档

版本 1.0 | 2026年9月 | 内部技术文档

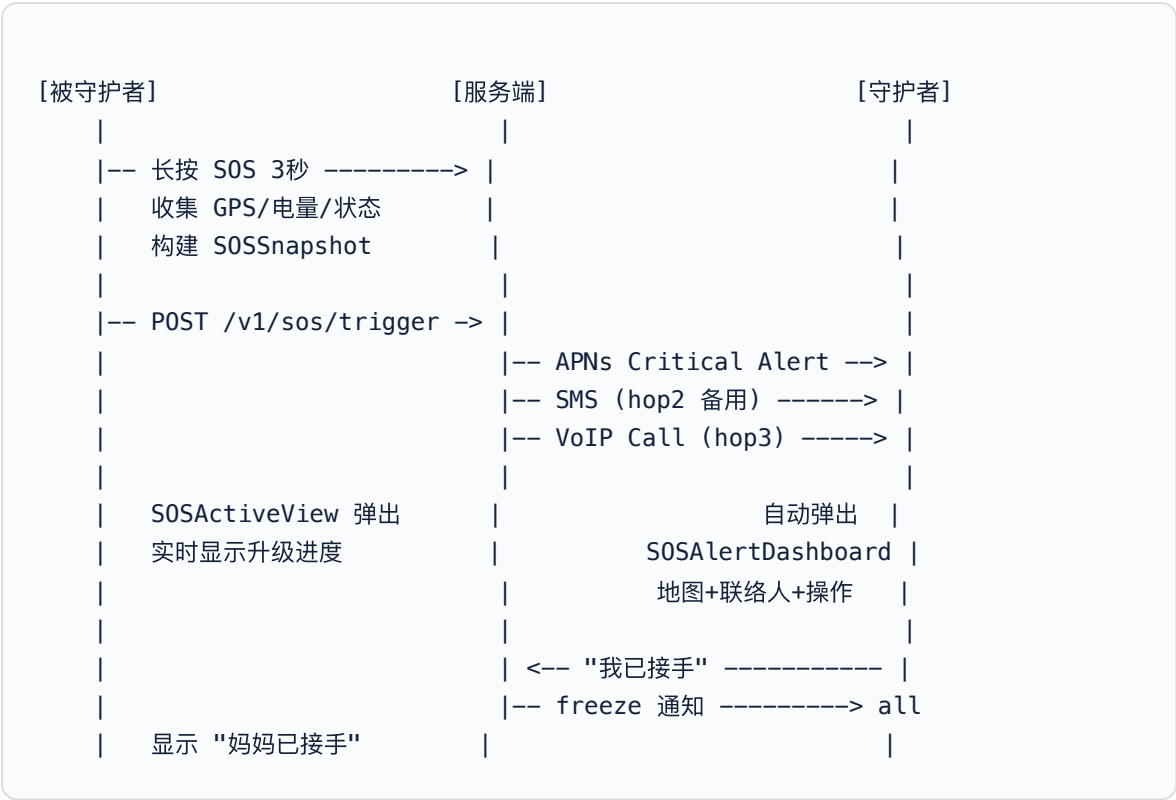
涵盖: 数据模型、iOS 推送架构、SOS 触发流程、警报面板、升级状态机

目录

- 系统总览
- 数据模型设计
- 紧急联络人录入流程
- SOS 触发与数据收集
- iOS 推送通知架构 (核心)
- 服务端推送架构
- 守护者警报面板 (SOSAlertDashboardView)
- 被守护者 SOS 进行中页
- 升级状态机集成
- 端到端数据流
- 离线与降级策略
- 安全与隐私
- 实现优先级

1. 系统总览

紧急提示系统是守灯 App 的核心功能链路。从被守护者按下 SOS 的瞬间起，系统必须在秒级内完成信息收集、推送、弹窗展示，让守护者能够立即掌握现场态势并采取行动。



设计原则

原则	说明
秒级响应	SOS 触发到守护者手机弹窗 < 3 秒 (APNs Critical Alert)
穿透免打扰	使用 iOS Critical Alert，即使手机静音/勿扰模式也会响铃振动
多通道冗余	推送 → SMS → 语音电话，三层升级，确保必达
离线可用	无网时本地缓存 SOS，恢复后自动上报；守护者离线时 SMS 兜底
一键冻结	"我已接手" 立即停止所有后续升级，防止惊动全家
态势感知	地图 + 当地资源 + 现场信息，让远在异国的守护者也能有效行动

2. 数据模型设计

2.1 SOSSnapshot — 紧急快照

SOS 触发瞬间的完整状态快照，打包发送给所有守护者。

```
struct SOSSnapshot: Codable {
    let sosEvent: SOSEvent           // 触发事件（时间、方式、位置）
    let protectedPerson: SnapshotPerson // 被守护者基本信息
    let lastLocation: Location?       // 最后已知 GPS 坐标
    let locationTrail: [Location]     // 最近 10 分钟位置轨迹
    let batteryLevel: Double?         // 0.0-1.0
    let batteryState: BatteryState    // charging/unplugged
    let localContacts: [LocalEmergencyContact] // 当地预录联络人
    let regionalInfo: RegionalEmergencyInfo // 当地急救号码
    let nearbyGuardians: [GuardianDistance] // 守护者按距离排序
    let safeZones: [SafeZone]         // 附近安全区域
    let lastCheckIn: Date?            // 最后一次报平安
    let lastPhoneActivity: Date?      // 最后一次手机交互
    let createdAt: Date
}

struct SnapshotPerson: Codable {
    let id: String
    let displayName: String
    let avatarInitial: String
    let countryCode: String
    let cityName: String
    let timeZone: String             // identifier
    let phone: String?              // 可直拨号码
}

struct GuardianDistance: Codable {
    let guardian: Guardian
    let distanceKm: Double?         // 与被守护者的直线距离
    let isOnDuty: Bool
}
```

2.2 LocalEmergencyContact — 扩展

在现有模型基础上增加类型、坐标、可用时间。

```
struct LocalEmergencyContact: Codable, Identifiable {
    let id: String
    var name: String
    var relationship: String           // "邻居", "社区医院", "大使馆"
    var type: ContactType             // .personal / .institution / .neighbor
    var phone: String
    var email: String
    var address: String
    var latitude: Double?             // 机构可标注地图
    var longitude: Double?
    var availableHours: String?       // "24h", "09:00-17:00"
    var notes: String?               // "会说中文", "5分钟可到"
    var countryCode: String

    enum ContactType: String, Codable {
        case personal           // 亲友、熟人
        case institution        // 医院、警局、大使馆
        case neighbor           // 邻居、房东、社区
    }
}
```

3. 紧急联络人录入流程

3.1 录入入口

角色	入口位置	说明
守护者	成员详情页 → "当地紧急资源"	为每位被守护者预录当地联络人
被守护者	设置 → "紧急联络人"	自己录入身边的联络人
守护者	首次绑定引导流	绑定被守护者时引导录入 1-3 个联络人

3.2 录入界面设计

分三个 Tab: 亲友熟人 当地机构 邻居/社区

亲友熟人: 姓名、关系、电话、备注 (如 "在当地, 开车 10 分钟可到")

当地机构: 名称、类型 (医院/警局/大使馆/学校)、电话、地址。使用 `MKLocalSearch` 搜索附近机构, 自动填入坐标和地址。

邻居/社区: 姓名、关系 (邻居/房东/物业/社区工作人员)、电话、楼栋门牌号。

3.3 存储策略

联络人按被守护者 ID 分别存储, 而非全局共享:

```
// LocalStore
func saveEmergencyContacts(_ contacts: [LocalEmergencyContact],
                           forPerson id: String) {
    save(contacts, filename: "emergency_contacts_\(id).json")
}

func loadEmergencyContacts(forPerson id: String) -> [LocalEmergencyContact] {
    load(filename: "emergency_contacts_\(id).json") ?? []
}
```

4. SOS 触发与数据收集

4.1 触发方式

方式	触发条件	优先级
长按 SOS 按钮	主屏长按 3 秒	用户主动
Apple Watch	表盘 complication 或摇晃手势	用户主动
蓝牙按钮	外接实体紧急按钮	用户主动
胁迫密码	输入预设的伪装密码解锁	静默触发
跌倒检测	加速度 + 陀螺仪算法判定	自动触发
语音唤醒词	"救命" 等自定义短语 (V3)	自动触发

4.2 触发瞬间的数据收集

```
// AppCoordinator.swift
func triggerSOS(method: SOSTriggerMethod) {
    // 1. 请求最精确的 GPS (kCLLocationAccuracyBest)
    locationManager.requestPreciseLocation()

    // 2. 读取当前电量
    let battery = UIDevice.current.batteryLevel
    let batteryState = UIDevice.current.batteryState

    // 3. 构建 SOSEvent
    let event = SOSEvent(
        id: UUID().uuidString,
        protectedPersonId: currentUser?.id ?? "",
        triggeredAt: Date(),
        triggerMethod: method,
        location: locationManager.lastKnownLocation,
        batteryLevel: Double(battery),
        escalationState: .initiated
    )

    // 4. 构建 SOSSnapshot
    let snapshot = SOSSnapshot(
        sosEvent: event,
        protectedPerson: buildSnapshotPerson(),
        lastLocation: locationManager.lastKnownLocation,
        locationTrail: locationManager.recentTrail(minutes: 10),
        batteryLevel: Double(battery),
        batteryState: mapBatteryState(batteryState),
        localContacts: localStore.loadEmergencyContacts(
            forPerson: currentUser?.id ?? ""
        ),
        regionalInfo: RegionalEmergencyInfo.forCountry(
            currentUser?.countryCode ?? "CN"
        ),
        nearbyGuardians: computeGuardianDistances(),
        safeZones: localStore.loadSafeZones(),
        lastCheckIn: ...,
        lastPhoneActivity: ...,
        createdAt: Date()
    )

    // 5. 本地持久化 (离线兜底)
    localStore.saveActiveSOSSnapshot(snapshot)
    activeSOSEvent = event

    // 6. 上报服务端 → 服务端负责推送
    Task {
        try await apiClient.post("/v1/sos/trigger", body: snapshot)
    }
}
```

```
// 7. 启动高频位置上报 (每 5 秒)
locationManager.startHighFrequencyTracking()

// 8. 启动升级状态机
escalationEngine.startEscalation(
    sosEvent: event,
    protectedPerson: ...,
    allGuardians: myGuardians
)

// 9. 触觉反馈
UINotificationFeedbackGenerator().notificationOccurred(.warning)
}
```

5. iOS 推送通知架构 (核心)

这是整个紧急系统的技术关键。iOS 推送有多种类型，针对紧急场景需要组合使用。

5.1 iOS 推送类型对比

类型	唤醒 App	穿透免打扰	显示通知	延迟	适用场景
普通推送 (Alert Push)	否	否	是	秒级	日常通知
Critical Alert (关键警报)	否	是	是 (有声音)	秒级	SOS 首选
Silent Push (静默推送)	是	否	否	不保证	后台数据同步
VoIP Push (PushKit)	是	是	CallKit UI	<1秒	Hop3 语音外呼
Notification Service Extension	Extension 可运行	跟随原推送	修改后显示	+几十ms	富内容修改

5.2 Critical Alert — SOS 的核心推送通道

Critical Alert 是什么？

iOS 提供的最高优先级通知类型。即使用户开启了「勿扰模式」「静音模式」甚至「睡眠专注模式」， Critical Alert 仍会发出声音和震动。这正是紧急安全 App 所需要的。

5.2.1 获取 Critical Alert 权限

Critical Alert 需要 Apple 的 **特殊授权 (Entitlement)**，不是所有 App 都能使用：

1. 在 [Apple Developer](#) 提交申请表
2. 说明 App 用途: 人身安全紧急警报 (Personal Safety Emergency Alert)
3. Apple 审核通过后，在 App ID 中启用 Critical Alerts 能力
4. 在 Xcode 的 entitlements 文件中添加：

```
<!-- ShouDeng.entitlements -->
<key>com.apple.developer.usernotifications.critical-alerts</key>
<true/>
```

5.2.2 客户端请求授权

```
import UserNotifications

func requestNotificationPermissions() {
    let center = UNUserNotificationCenter.current()
    center.requestAuthorization(
        options: [.alert, .sound, .badge, // 普通权限
                 .criticalAlert]          // Critical Alert 权限
    ) { granted, error in
        if granted {
            // 用户同意 → 注册远程推送
            DispatchQueue.main.async {
                UIApplication.shared.registerForRemoteNotifications()
            }
        }
    }
}
```

用户授权体验: iOS 会弹出两次授权窗口 — 一次为普通通知，一次专门为 Critical Alert。Critical Alert 的弹窗会明确告知用户 "即使设备静音或开启勿扰模式，此 App 仍可发出声音"。用户可以单独关闭 Critical Alert 而保留普通通知。

5.2.3 Device Token 注册

```
// AppDelegate.swift
func application(_ application: UIApplication,
    didRegisterForRemoteNotificationsWithDeviceToken deviceToken: Data) {
    // 将 token 转为 hex string
    let token = deviceToken.map { String(format: "%02x", $0) }.joined()

    // 上传到服务端，绑定当前用户
    Task {
        try await apiClient.put(
            "/v1/user/me/push-token",
            body: ["token": token, "platform": "ios"]
        )
    }
}
```

5.2.4 APNs Payload — Critical Alert 格式

```
{
  "aps": {
    "alert": {
      "title": "小雨正在求助",
      "subtitle": "基辅市中心 | 手动触发",
      "body": "电量 12% | 4 分钟前最后活动 | 点击查看地图和当地资源"
    },
    "sound": {
      "critical": 1,           // 标记为 Critical Alert
      "name": "sos_alarm.caf", // 自定义紧急音效
      "volume": 1.0           // 系统最大音量 (0.0-1.0)
    },
    "interruption-level": "critical",
    "relevance-score": 1.0,
    "category": "SOS_ALERT",   // 关联通知操作按钮
    "thread-id": "sos_event_xxx",
    "mutable-content": 1       // 触发 Notification Service Extension
  },
  "sos_snapshot": {           // 自定义数据
    "event_id": "sos_abc123",
    "person_id": "user_xiaoyu",
    "person_name": "小雨",
    "latitude": 50.4501,
    "longitude": 30.5234,
    "battery": 0.12,
    "trigger_method": "longPress",
    "escalation_hop": "1",
    "snapshot_url": "https://api.shoudeng.app/v1/sos/sos_abc123/snapshot"
  }
}
```

Payload 大小限制: APNs payload 最大 4KB。SOSSnapshot 完整数据可能超过此限制，因此 payload 中只放关键字段 (位置、电量、人名)，完整 snapshot 通过 snapshot_url 在 App 打开后异步拉取。Notification Service Extension 也可以在后台预拉取。

5.2.5 Notification Service Extension — 后台预处理

在通知显示给用户之前，Extension 有约 30 秒的后台运行时间，用于：

- 拉取完整 SOSSnapshot (通过 snapshot_url)
- 反向地理编码 (坐标 → 地址文字)
- 缓存地图缩略图
- 本地化通知内容

```

// NotificationService.swift (Notification Service Extension target)
class NotificationService: UNNotificationServiceExtension {

    override func didReceive(
        _ request: UNNotificationRequest,
        withContentHandler handler: @escaping (UNNotificationContent) -> Void
    ) {
        guard let content = request.content.mutableCopy()
            as? UNMutableNotificationContent else { return }

        let userInfo = request.content.userInfo

        // 1. 拉取完整 snapshot
        if let urlStr = userInfo["snapshot_url"] as? String,
            let url = URL(string: urlStr) {
            let data = try? Data(contentsOf: url)
            // 存入 App Group 共享容器, 主 App 打开后可直接读取
            if let data {
                let container = FileManager.default
                    .containerURL(forSecurityApplicationGroupIdentifier:
                        "group.com.shoudeng.app")!
                try? data.write(to: container
                    .appendingPathComponent("latest_sos_snapshot.json"))
            }
        }

        // 2. 增强通知内容
        if let lat = userInfo["latitude"] as? Double,
            let lon = userInfo["longitude"] as? Double {
            // 反向地理编码
            let geocoder = CLGeocoder()
            let location = CLLocation(latitude: lat, longitude: lon)
            geocoder.reverseGeocodeLocation(location) { places, _ in
                if let place = places?.first {
                    content.subtitle = place.name ?? content.subtitle
                }
                handler(content)
            }
            return
        }

        handler(content)
    }
}

```

5.2.6 Notification Actions — 通知上的快捷操作

```
// 注册通知分类和操作按钮
func registerNotificationCategories() {
    let takeOverAction = UNNotificationAction(
        identifier: "TAKE_OVER",
        title: "我已接手",
        options: [.authenticationRequired, .foreground]
    )
    let callAction = UNNotificationAction(
        identifier: "CALL_PERSON",
        title: "立即拨号",
        options: [.foreground]
    )
    let viewAction = UNNotificationAction(
        identifier: "VIEW_MAP",
        title: "查看地图",
        options: [.foreground]
    )

    let sosCategory = UNNotificationCategory(
        identifier: "SOS_ALERT",
        actions: [takeOverAction, callAction, viewAction],
        intentIdentifiers: [],
        options: [.customDismissAction]
    )

    UNUserNotificationCenter.current()
        .setNotificationCategories([sosCategory])
}
```

5.3 Silent Push — 后台数据同步

用于 SOS 期间的实时位置更新同步 (非紧急, 补充数据):

```
// Silent push payload
{
  "aps": {
    "content-available": 1    // 标记为 silent push
  },
  "type": "location_update",
  "sos_id": "sos_abc123",
  "latitude": 50.4505,
  "longitude": 30.5230,
  "timestamp": "2026-09-12T14:07:05Z"
}
```

Silent Push 限制: iOS 不保证 silent push 的即时送达。系统会根据电量、用户使用习惯等 动态调节频率。如果 App 被系统杀死, silent push 可能不会唤醒 App。因此 silent push 只用于补充数据, 不能作为 SOS 的主通道。

5.4 VoIP Push (PushKit) — Hop3 语音外呼

用于升级链第三步: 自动语音拨打守护者电话。

```

import PushKit

class VoIPPushHandler: NSObject, PKPushRegistryDelegate {

    private let registry = PKPushRegistry(queue: .main)

    func setup() {
        registry.delegate = self
        registry.desiredPushTypes = [.voIP]
    }

    // 收到 VoIP token
    func pushRegistry(_ registry: PKPushRegistry,
        didUpdate pushCredentials: PKPushCredentials,
        for type: PKPushType) {
        let token = pushCredentials.token
            .map { String(format: "%02x", $0) }.joined()
        // 上传 VoIP token 到服务端
        Task {
            try await apiClient.put("/v1/user/me/voip-token",
                body: ["token": token])
        }
    }

    // 收到 VoIP 推送 → 必须立即弹出 CallKit UI
    func pushRegistry(_ registry: PKPushRegistry,
        didReceiveIncomingPushWith payload: PKPushPayload,
        for type: PKPushType,
        completion: @escaping () -> Void) {

        let data = payload.dictionaryPayload
        let callerName = data["person_name"] as? String ?? "紧急求助"

        // 必须调用 CallKit reportNewIncomingCall
        // 否则 iOS 会终止 App (iOS 13+ 强制要求)
        CallKitManager.shared.reportIncomingCall(
            from: callerName,
            sosEventId: data["sos_id"] as? String
        )

        completion()
    }
}

```

VoIP Push 重要限制 (iOS 13+):

1. 收到 VoIP push 后**必须**立即调用 `CXProvider.reportNewIncomingCall()` , 否则 iOS 会杀死 App 并可能吊销推送权限
2. Apple 审核时会检查 VoIP push 是否真正用于通话场景

3. 守灯的 Hop3 自动外呼属于合理的 VoIP 使用: 服务端通过 Twilio 发起真实语音通话, VoIP push 用于唤醒 App 显示来电界面

5.5 Xcode 工程配置

Required Capabilities (项目 → Signing & Capabilities)

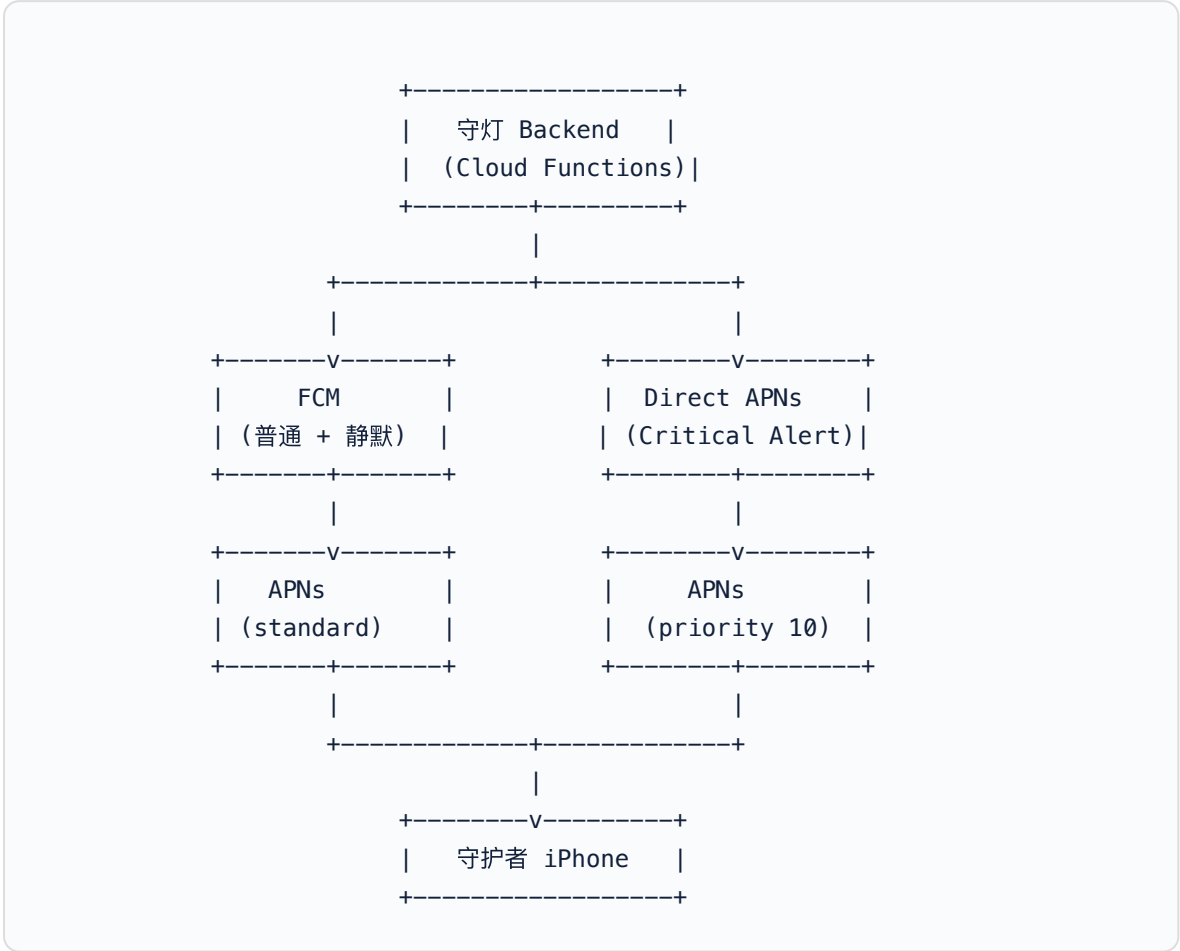
Capability	用途
Push Notifications	基础推送
Background Modes → Remote notifications	Silent push 后台唤醒
Background Modes → Voice over IP	VoIP push (PushKit)
Background Modes → Location updates	SOS 期间高频 GPS
App Groups	主 App 与 Notification Extension 共享数据

Entitlements 文件

```
<?xml version="1.0" encoding="UTF-8"?>
<plist version="1.0">
<dict>
  <key>aps-environment</key>
  <string>production</string>
  <key>com.apple.developer.usernotifications.critical-alerts</key>
  <true/>
  <key>com.apple.security.application-groups</key>
  <array>
    <string>group.com.shoudeng.app</string>
  </array>
</dict>
</plist>
```

6. 服务端推送架构

6.1 架构选择: Firebase Cloud Messaging (FCM) + 直连 APNs



6.2 为什么需要直连 APNs?

通道	方式	原因
日常通知	FCM	简单易用, FCM 自动管理 token 映射、topic 订阅、Android 兼容
Critical Alert	直连 APNs	FCM 不完全支持 Critical Alert 的 <code>sound.critical</code> 字段。直连 APNs 可完全控制 payload 格式, 确保 <code>interruption-level: critical</code> 正确送达
VoIP Push	直连 APNs	VoIP push 使用独立的 PushKit token, 不经过 FCM

6.3 APNs 认证: Token-Based (p8 密钥)

```
// 服务端 (Node.js / Cloud Functions)
const apn = require('@parse/node-apn');

const provider = new apn.Provider({
  token: {
    key: './AuthKey_XXXXXXXXXX.p8', // Apple Developer 下载
    keyId: 'XXXXXXXXXX',           // Key ID
    teamId: 'YYYYYYYYYY',         // Team ID
  },
  production: true
});

async function sendCriticalSOSAlert(deviceToken, snapshot) {
  const note = new apn.Notification();

  note.expiry = Math.floor(Date.now() / 1000) + 300; // 5分钟内送达
  note.priority = 10;                                // 立即送达
  note.pushType = 'alert';
  note.topic = 'com.shoudeng.app';

  note.aps = {
    'alert': {
      'title': `${snapshot.person_name}正在求助`,
      'subtitle': `${snapshot.city} | ${snapshot.trigger_method}`,
      'body': `电量 ${Math.round(snapshot.battery*100)}%`
    },
    'sound': {
      'critical': 1,
      'name': 'sos_alarm.caf',
      'volume': 1.0
    },
    'interruption-level': 'critical',
    'relevance-score': 1.0,
    'category': 'SOS_ALERT',
    'mutable-content': 1
  };

  note.payload = {
    sos_snapshot: {
      event_id: snapshot.event_id,
      person_id: snapshot.person_id,
      person_name: snapshot.person_name,
      latitude: snapshot.latitude,
      longitude: snapshot.longitude,
      battery: snapshot.battery,
      trigger_method: snapshot.trigger_method,
      escalation_hop: '1',
      snapshot_url: `https://api.shoudeng.app/v1/sos/${snapshot.event_id}/snaps
    }
  };
}
```

```
const result = await provider.send(note, deviceToken);
return result;
}
```

6.4 服务端 SOS 处理流程

```
// Cloud Function: onSOSTrigger
exports.onSOSTrigger = functions.https.onCall(async (data, context) => {
  const snapshot = data.snapshot;
  const protectedPersonId = snapshot.sosEvent.protectedPersonId;

  // 1. 存入 Firestore
  await db.collection('sos_events').doc(snapshot.sosEvent.id).set(snapshot);

  // 2. 查询所有守护者
  const guardians = await getGuardiansForPerson(protectedPersonId);

  // 3. Hop 1: 找到值班守护者, 发 Critical Alert
  const onDuty = guardians.find(g => g.isOnDuty) || guardians[0];
  if (onDuty) {
    const token = await getAPNsToken(onDuty.id);
    await sendCriticalSOSAlert(token, snapshot);
  }

  // 4. 启动升级定时器
  // Hop 2: 3分钟后如无人应答 → 通知所有守护者 + SMS
  await scheduleEscalation(snapshot.sosEvent.id, 'hop2', 180);

  // Hop 3: 5分钟后如仍无人应答 → VoIP 语音外呼
  await scheduleEscalation(snapshot.sosEvent.id, 'hop3', 300);

  return { success: true, eventId: snapshot.sosEvent.id };
});
```

6.5 SMS 兜底 (Hop 2)

通过 Twilio 发送 SMS, 确保即使 App 未安装或推送失败也能通知到:

```
const twilio = require('twilio')(accountSid, authToken);

async function sendSOSSMS(phoneNumber, personName, location) {
  await twilio.messages.create({
    body: `[守灯紧急] ${personName}正在求助! `
      + `位置: ${location}。`
      + `打开守灯App查看详情, 或回复"接手"确认处理。`,
    from: '+1XXXXXXXXXX',
    to: phoneNumber
  });
}
```

7. 守护者警报面板 (SOSAlertDashboardView)

7.1 客户端接收推送后的处理

```
// AppDelegate.swift
func userNotificationCenter(
    _ center: UNUserNotificationCenter,
    didReceive response: UNNotificationResponse,
    withCompletionHandler handler: @escaping () -> Void
) {
    let userInfo = response.notification.request.content.userInfo

    switch response.actionIdentifier {
    case "TAKE_OVER":
        // 直接冻结升级链
        coordinator.handleTakeOver(
            sosId: userInfo["event_id"] as? String
        )

    case "CALL_PERSON":
        // 拨打被守护者电话
        coordinator.callProtectedPerson(
            sosId: userInfo["event_id"] as? String
        )

    default:
        // 点击通知本体 → 打开 App → 弹出警报面板
        coordinator.openSOSAlert(
            sosId: userInfo["event_id"] as? String
        )
    }

    handler()
}
```

7.2 自动弹出机制

```
// RootView.swift
struct RootView: View {
    @EnvironmentObject var coordinator: AppCoordinator

    var body: some View {
        ZStack {
            // ... 正常 Tab 界面
        }
        .fullScreenCover(
            item: $coordinator.activeSOSEvent
        ) { event in
            SOSAlertDashboardView(sosEvent: event)
                .environmentObject(coordinator)
                .interactiveDismissDisabled() // 不可下滑关闭
        }
    }
}
```

7.3 SOSAlertDashboardView 页面布局

区域	内容	数据来源
红色横幅	"小雨正在求助" + 触发方式 + 实时计时器	SOSEvent
地图	MapKit Map 显示: - 红色脉冲 pin: 被守护者最后位置 - 蓝色 pin: 附近机构联络人 - 绿色圆圈: 安全区域 - 紫色 pin: 守护者位置 - 虚线: 最近移动轨迹	SOSSnapshot
操作按钮	"立即拨号" (打给被守护者) + "我已接手" (冻结升级链)	EscalationEngine
现场信息卡	电量 + 电量趋势、GPS 精度、最后活动时间、触发方式、是否在安全区	SOSSnapshot
当地资源 (3 组)	1. 熟人亲友 (可拨号) 2. 当地机构 (医院/警局, 可拨号+地图导航) 3. 官方急救号码 (按国家自动匹配)	LocalEmergencyContact RegionalEmergencyInfo
升级链状态	3-hop 实时进度 (通知 → 全员 → 语音外呼)	EscalationEngine.Session

7.4 地图实现

```
import MapKit

struct SOSMapView: View {
    let snapshot: SOSSnapshot

    var body: some View {
        Map {
            // 被守护者最后位置 (红色脉冲)
            if let loc = snapshot.lastLocation {
                Annotation("求助位置",
                    coordinate: CLLocationCoordinate2D(
                        latitude: loc.latitude,
                        longitude: loc.longitude)) {
                    SOSPulsingPin() // 自定义脉冲动画
                }
            }

            // 附近机构 (蓝色 pin)
            ForEach(institutionContacts) { contact in
                if let lat = contact.latitude,
                    let lon = contact.longitude {
                    Annotation(contact.name,
                        coordinate: CLLocationCoordinate2D(
                            latitude: lat, longitude: lon)) {
                        InstitutionPin(contact: contact)
                    }
                }
            }

            // 安全区域 (绿色圆圈)
            ForEach(snapshot.safeZones) { zone in
                MapCircle(
                    center: CLLocationCoordinate2D(
                        latitude: zone.latitude,
                        longitude: zone.longitude),
                    radius: zone.radius
                )
                .foregroundColor(.green.opacity(0.15))
                .stroke(.green, lineWidth: 1)
            }

            // 移动轨迹 (虚线)
            if !snapshot.locationTrail.isEmpty {
                MapPolyline(coordinates: trailCoordinates)
                    .stroke(.red.opacity(0.5), lineWidth: 2)
            }
        }
        .mapStyle(.standard(elevation: .flat))
    }
}
```

8. 被守护者 SOS 进行中页

8.1 SOSActiveView 改造

当前 SOSActiveView 使用硬编码 demo 数据，需接入真实 SOS 状态:

信息	来源
触发时间 + 持续计时	<code>SOSEvent.triggeredAt</code>
触发位置	<code>SOSEvent.location</code> (反向地理编码)
升级进度	<code>EscalationEngine.activeSession</code>
守护者响应状态	WebSocket 实时推送 / 轮询
"妈妈已接手"	<code>EscalationEngine.Session.frozenBy</code>

8.2 实时状态同步

SOS 期间，被守护者端需实时知道谁已读、谁接手。两种方案:

方案	优点	缺点
Firestore 实时监听	简单可靠，Firebase SDK 自带	依赖网络，费用随监听数增长
WebSocket	延迟最低 (<100ms)，灵活	需自建 WebSocket 服务

推荐: **Firestore 实时监听** (MVP 阶段)，SOS 事件作为一个文档，所有相关方监听同一文档变化:

```
// 被守护者端: 监听 SOS 事件变化
let listener = db.collection("sos_events")
    .document(sosEventId)
    .addSnapshotListener { snapshot, error in
        guard let data = snapshot?.data() else { return }

        // 更新 UI: 谁已读、谁接手
        if let frozenBy = data["frozen_by"] as? String {
            // 显示 "妈妈已接手"
            showFrozenBanner(guardianName: frozenBy)
        }

        // 更新升级链步骤状态
        if let hop = data["current_hop"] as? Int {
            updateEscalationUI(hop: hop)
        }
    }
}
```

9. 升级状态机集成

9.1 现有 EscalationEngine 与推送的对接

触发 SOS



Hop 1: onDutyGuardian (Critical Alert → APNs)

| 等待 3 分钟 (夜间 1.5 分钟)
| 无应答?



Hop 2: allGuardians (Critical Alert + SMS → APNs + Twilio)

| 等待 2 分钟
| 无应答?



Hop 3: voiceCall (VoIP Push → PushKit + Twilio Voice)

| 每人最多拨 3 次, 间隔 30 秒
| 全部未接?



EXHAUSTED → 通知被守护者 "所有守护者暂无法联系"

→ 显示当地急救号码一键拨打

任何一步: 守护者点 "我已接手"

→ freeze() → 立即停止所有后续动作
→ 通知所有人 "已有人接手"

9.2 Callback 与推送通道映射

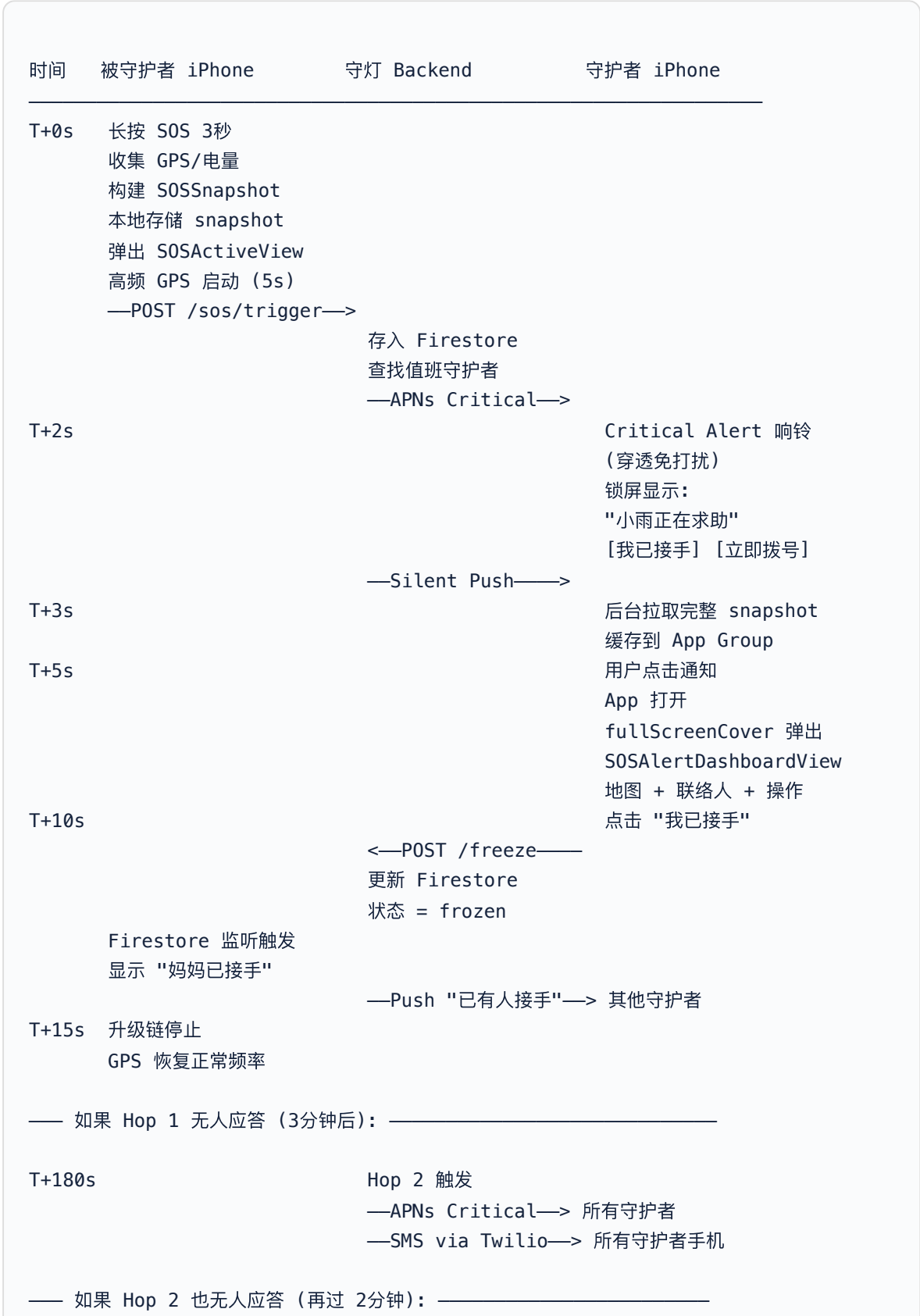
```
// AppCoordinator 中配置 EscalationEngine 回调
func setupEscalationCallbacks() {
    escalationEngine.onSendNotification = { request in
        Task {
            switch request.priority {
            case .critical:
                // 通过服务端发 Critical Alert (直连 APNs)
                try await self.apiClient.post(
                    "/v1/push/critical", body: request)
            case .high:
                // 通过 FCM 发普通高优先级推送
                try await self.apiClient.post(
                    "/v1/push/send", body: request)
            case .normal:
                // 通过 FCM 发普通推送
                try await self.apiClient.post(
                    "/v1/push/send", body: request)
            }
        }
    }

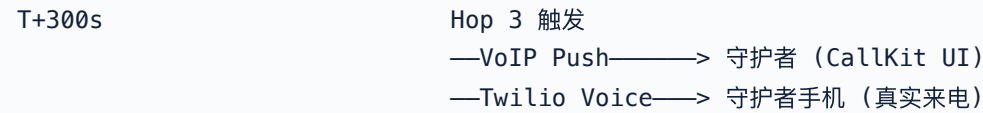
    escalationEngine.onInitiateVoiceCall = { guardianId, sosId in
        Task {
            // 服务端通过 Twilio 发起语音外呼
            // 同时发 VoIP push 唤醒守护者 App
            try await self.apiClient.post(
                "/v1/sos/\(sosId)/voice-call",
                body: ["guardian_id": guardianId])
        }
    }

    escalationEngine.onEscalationFrozen = { session, guardianId in
        // 更新 Firestore SOS 文档
        Task {
            try await self.apiClient.post(
                "/v1/sos/\(session.sosEvent.id)/freeze",
                body: ["guardian_id": guardianId])
        }
    }
}
```

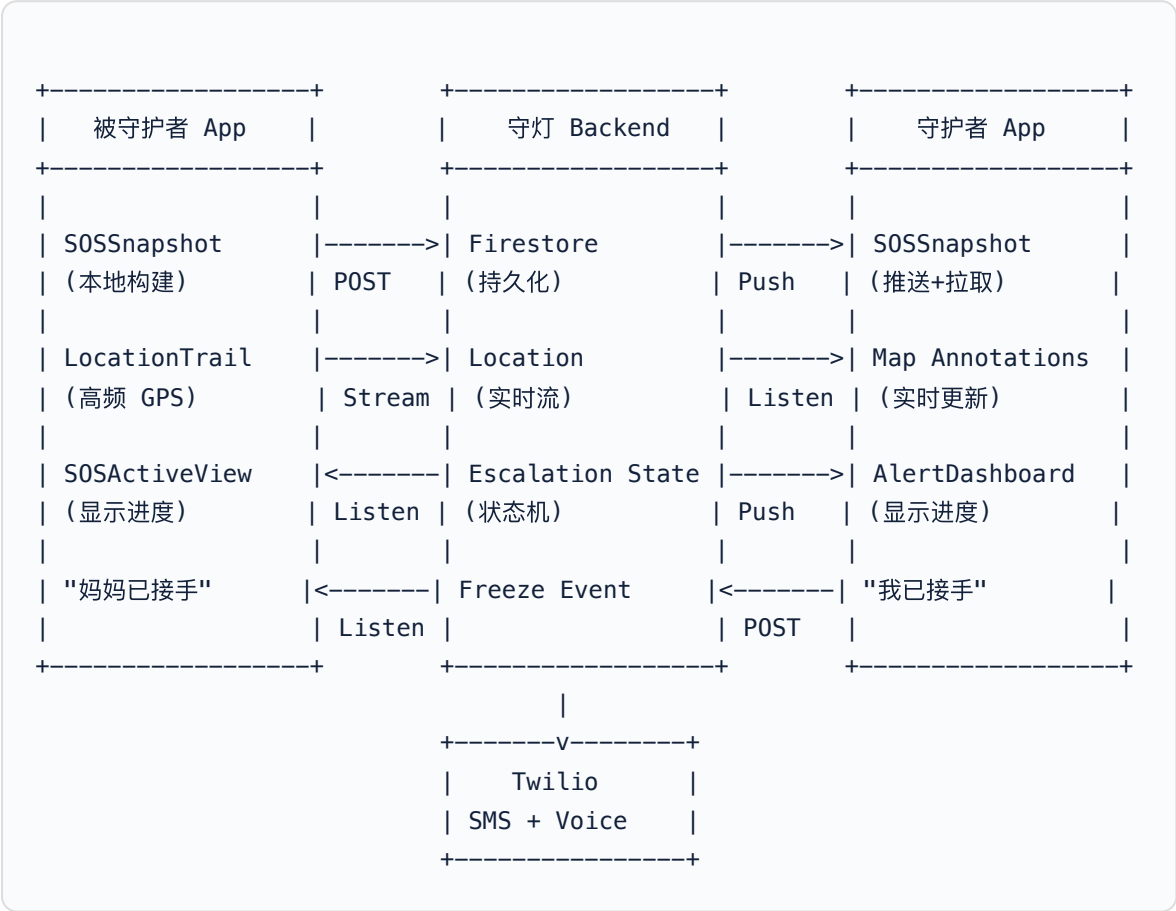
10. 端到端数据流

10.1 完整时序图





10.2 数据流向图



11. 离线与降级策略

11.1 被守护者离线

场景	策略
SOS 时无网络	本地存储 SOSSnapshot + 加入 OfflineQueue 。 恢复网络后自动上报 (已有 OfflineQueue 机制)
GPS 无信号	使用最后已知位置 + WiFi/基站定位。 Location.accuracy 标注精度供守护者参考
手机关机/没电	服务端检测心跳中断 → 自动升级为 "失联" 状态 → 通知守护者

11.2 守护者离线

场景	策略
App 未安装	SMS 兜底 (Hop 2)
手机关机	APNs 会暂存推送，开机后送达。 同时 SMS + 语音外呼确保备用通道
网络差	Notification Service Extension 预拉取 snapshot 并缓存。 App 打开后优先读缓存

11.3 服务端降级

场景	策略
Firebase 宕机	直连 APNs 不经过 FCM，Critical Alert 仍可送达。 Twilio SMS/Voice 独立于 Firebase
API 超时	客户端本地运行 EscalationEngine ， 以 P2P 方式发送通知 (需要守护者设备的 push token 本地缓存)

12. 安全与隐私

措施	说明
传输加密	所有 API 通信使用 TLS 1.3。APNs 原生 TLS
存储加密	SOSSnapshot 本地使用 iOS Data Protection (NSFileProtectionComplete)
位置数据	GPS 轨迹仅在 SOS 期间收集，事件结束后按保留策略清除
推送内容	Critical Alert 中不包含精确坐标文字（防止锁屏泄露）。 仅显示城市级位置，详细信息需解锁后在 App 内查看
VoIP Token	VoIP push token 和 APNs token 分别管理，服务端加密存储
SMS 内容	SMS 不包含精确位置，仅提示 "打开 App 查看"

13. 实现优先级

Phase 1: 核心链路 (MVP)

#	任务	文件
1	SOSSnapshot 模型	Events.swift
2	triggerSOS() 构建 snapshot	AppCoordinator.swift
3	新建 SOSAlertDashboardView (地图+联络人+操作)	新文件
4	RootView fullScreenCover 自动弹出	RootView.swift
5	Notification Service Extension target	新 target
6	Critical Alert 权限申请 + 客户端授权	Entitlements + PushService.swift
7	服务端: APNs 直连 Critical Alert 发送	Cloud Function

Phase 2: 升级链

#	任务	文件
8	EscalationEngine callback → 服务端推送	AppCoordinator.swift
9	Hop 2: SMS via Twilio	Cloud Function
10	Hop 3: VoIP Push + Twilio Voice + CallKit	新文件 + Cloud Function
11	Firestore 实时监听 (双端同步)	AppCoordinator.swift

Phase 3: 当地资源

#	任务	文件
12	扩展 LocalEmergencyContact 模型 + per-person 存储	EmergencyContactsView.swift , LocalStore.swift
13	机构搜索 (MKLocalSearch)	EmergencyContactsView.swift
14	SOSActiveView 接入真实数据	ProtectedHomeView.swift
15	离线队列串联	OfflineQueue.swift